Guía del proyecto de alumnos

Explicación de los archivos y del funcionamiento de PHP con MySQL

Esta guía permite presentar el programa desde la lista inicial hasta el guardado de cursos, la edición y la eliminación. Describe el código que queda en la entrega y cómo restaurar la base de datos con su contenido actual.

Cómo se organiza

El navegador solicita una página PHP a Apache. PHP comprueba la sesión, consulta MySQL mediante PDO y devuelve HTML. En el formulario, JavaScript agrega campos y pide sugerencias de cursos sin recargar la página.

Archivo PHP	Responsabilidad principal
index.php	Lista de estudiantes y enlaces de las acciones.
login.php / logout.php	Ingreso y cierre de sesión.
ver.php	Consulta pública de un alumno y sus cursos.
nuevo.php	Entrada al formulario de alta.
editar_db.php	Entrada al formulario de edición.
alumno_form.php	Campos, validación y guardado de altas y ediciones.
delete.php	Confirmación y eliminación de un alumno.
cursos.php	Sugerencias del catálogo en formato JSON.
app.php	Funciones comunes de sesión, seguridad y presentación.
db.php	Conexión PDO a la base alumnos.
head.php	Estructura HTML inicial y hojas de estilo.

Reglas de acceso

Sin sesión: se ve la lista, el enlace Ingresar y la consulta Ver. **Con sesión:** se habilitan Nuevo, Editar y Eliminar. Las rutas protegidas también validan el acceso en el servidor; esconder enlaces por sí solo no protege una operación.

Dirección local: http://localhost/php_js_jquery/. Usuario de la aplicación: **admin@alumnos.local**. Contraseña inicial: **Alumno2026!**.

1 Lista consulta e inicio de sesión

index.php

Incluye app.php y db.php. Consulta los alumnos, ordenados del más reciente al más antiguo, y construye la tabla con Nombres, Apellidos, Correo, Aficiones y Acción. Los textos se imprimen con e() para que se muestren como datos. Si no hay registros, presenta un mensaje.

Para explicarlo: Es la pantalla principal. Según autenticado(), muestra Ingresar y Modo lectura, o el nombre del usuario, Salir y Nuevo. Cada fila conserva el id_alumno en los enlaces Ver, Editar y Eliminar, de modo que la selección se hace desde la lista.

login.php

Con GET muestra el formulario. Con POST valida el token CSRF, recibe correo y contraseña, busca el usuario por correo con una consulta preparada y verifica la contraseña mediante password_verify(). Si las credenciales no coinciden, muestra un error.

Para explicarlo: El correo identifica al usuario de la aplicación. La contraseña introducida se compara con un hash, no con texto plano. Cuando el acceso es correcto, se regenera el identificador de sesión, se guardan id_usuario y nombre, y se redirige a index.php.

logout.php

Recibe el formulario Salir de index.php. Exige POST, valida CSRF, vacía los datos de la sesión y regenera su identificador antes de regresar a la lista. No necesita abrir una conexión con MySQL.

Para explicarlo: Al salir se elimina id_usuario de la sesión. Por eso autenticado() deja de ser verdadero y las rutas de escritura vuelven a enviar al usuario al login.

ver.php

Obtiene el alumno indicado por id_alumno y sus cursos mediante las funciones compartidas. Muestra los cuatro datos del estudiante y una tabla con año y nombre de curso; permite regresar a la lista.

Para explicarlo: Esta pantalla es de lectura y está disponible sin autenticación. Los cursos se obtienen uniendo alumno_cursos con cursos. Si el id no existe, se regresa a index.php con un mensaje.

Qué diferencia hay entre GET y POST

GET solicita la lista, una consulta o un formulario. POST envía datos para autenticar, guardar, cerrar sesión o confirmar una eliminación. Abrir el enlace Eliminar con GET no borra el registro.

2 Alta y edición de estudiantes

nuevo.php

Carga las funciones comunes, exige autenticación y abre la conexión. Define $editar = false y carga alumno_form.php.

Para explicarlo: Este archivo selecciona el modo de creación. El formulario comienza vacío y, al grabar, la operación principal es INSERT INTO alumno. El id_usuario se obtiene de la sesión, no de una caja de texto.

editar_db.php

Carga app.php, comprueba la sesión y carga db.php. Define $editar = true e incluye el mismo formulario. El alumno se identifica por el id_alumno del enlace de la lista.

Para explicarlo: Es la única ruta de edición de la entrega. Con GET carga el alumno y sus cursos. Con POST actualiza ese alumno y después vuelve a la lista, conservando su id y su usuario asociado.

alumno_form.php

Es una vista y un procesador compartido. No se usa como una pantalla independiente. Verifica que existan $pdo y $editar; si se intenta abrir directamente sin ese contexto, devuelve 404. Evita repetir el HTML y la validación de Nuevo y Editar.

Validación: exige nombres, apellidos, correo y aficiones; controla sus longitudes y el formato del correo. Recibe cursos como un arreglo: cursos[índice][nombre] y cursos[índice][anio]. Ignora filas totalmente vacías, rechaza filas incompletas, años fuera de 1900 a 2100, nombres mayores de 50 caracteres y cursos repetidos en el mismo año. Acepta hasta 100 filas.

Guardado: inicia una transacción. En modo nuevo inserta el alumno y obtiene su id con lastInsertId(). En modo edición actualiza el alumno, elimina sus asignaciones anteriores y vuelve a insertar las recibidas. Esto permite modificar, agregar o quitar cursos desde el mismo formulario.

Por cada nombre de curso, el catálogo inserta uno nuevo o reutiliza el id existente mediante ON DUPLICATE KEY UPDATE. Después inserta la relación entre alumno, curso y año. La clave primaria impide repetir esa combinación, incluso cuando MySQL considera equivalentes dos nombres por su intercalación.

Si todo termina bien, commit() confirma los cambios, se prepara el mensaje de éxito y se redirige a index.php. Si ocurre un error de base de datos, rollBack() evita dejar un guardado parcial y el formulario conserva lo escrito para corregirlo.

Para explicarlo: el mismo formulario resuelve dos operaciones. La variable $editar cambia la carga inicial y la consulta principal, mientras que los campos, los controles y las reglas se reutilizan.

3 Funciones comunes conexión y cabecera

app.php

Inicia la sesión con cookies HttpOnly, SameSite=Lax y modo estricto. En HTTPS activa también el atributo Secure. Después define las funciones reutilizadas por las pantallas. Todas tienen llamadas en el proyecto.

Función	Qué hace y dónde se utiliza
e()	Escapa texto para HTML. Se usa al imprimir datos, valores y mensajes.
redirect()	Envía Location y termina la ejecución con exit.
autenticado()	Comprueba si existe id_usuario en la sesión. Se usa en la lista y el login.
requiere_login()	Redirige al login si falta la sesión. Se usa antes de las operaciones protegidas.
csrf()	Genera o recupera el token aleatorio guardado en la sesión.
validar_csrf()	Compara el token de POST con el de sesión mediante hash_equals(); rechaza con 403 si no coincide.
inicio()	Carga head.php, consume mensajes de sesión y muestra el título.
fin()	Cierra main, body y html. Se usa al final de las vistas.
buscar_alumno()	Valida el id de GET y consulta el alumno. Si falta, regresa a la lista.
cursos_alumno()	Une las asignaciones con el catálogo y devuelve año y nombre ordenados.

db.php

Crea $pdo con host 127.0.0.1, puerto 3306, base alumnos, usuario root y contraseña vacía. Activa utf8mb4, excepciones, resultados asociativos y consultas preparadas nativas.

Para explicarlo: PDO es el acceso de PHP a MySQL. prepare() define el SQL y execute() envía sus valores separados. La conexión es compartida mediante require_once para evitar cargarla dos veces en una solicitud.

head.php

Abre el documento HTML en español, establece UTF-8 y la adaptación al ancho de pantalla, imprime el título y enlaza Bootstrap y app.css. Abre el contenedor principal que fin() cerrará.

Para explicarlo: Es una plantilla común incluida por inicio(). Mantiene el diseño consistente sin copiar la cabecera en cada vista.

4 Eliminación y autocompletado

delete.php

Exige sesión, abre la conexión y busca al alumno elegido desde la lista. Con GET muestra su nombre, correo y la confirmación. Con POST valida CSRF, ejecuta DELETE FROM alumno WHERE id_alumno = ?, prepara el mensaje y redirige a index.php.

Para explicarlo: El enlace selecciona el registro, pero el borrado ocurre al confirmar el formulario. La relación ON DELETE CASCADE elimina sus filas de alumno_cursos. El catálogo cursos se conserva, porque puede ser utilizado por otros alumnos.

cursos.php

Es un endpoint de datos, no una vista HTML. Exige sesión, recibe term por GET, busca nombres que empiezan por ese texto y devuelve un arreglo JSON con hasta 15 sugerencias ordenadas. Un término vacío o mayor de 50 caracteres devuelve [].

Para explicarlo: Por ejemplo, escribir Cis puede encontrar Cisco Packet Tracer si ese curso ya está guardado. La consulta utiliza LIKE con un prefijo. Los signos % y _ escritos por la persona se escapan para buscarlos literalmente.

assets/cursos.js

Se ejecuta cuando el formulario está listo. Cuenta las filas iniciales y mantiene un contador para crear identificadores nuevos. El botón + añade una fila con año, nombre y botón para quitar; es type="button", por lo que no envía el formulario. La eliminación de filas se atiende con un evento delegado, que también funciona en las filas recién agregadas.

Cada caja de nombre activa autocomplete() de jQuery UI. Desde el primer carácter, después de una espera de 200 milisegundos, realiza una petición AJAX a cursos.php enviando term. El servidor devuelve JSON y el widget muestra las sugerencias. Seleccionar una copia el nombre al campo. También puede escribirse un curso nuevo y guardarlo con el alumno.

Si falla la consulta, limpia las sugerencias y muestra un mensaje. Cuando la respuesta corresponde a un login por sesión vencida, envía al usuario a login.php. El límite de 100 filas se verifica tanto en JavaScript como en PHP.

Para explicarlo: JavaScript modifica el formulario y solicita sugerencias; PHP valida y consulta la base. Agregar una caja no guarda nada todavía. El registro y los cursos se guardan al presionar Grabar.

Ruta de una sugerencia

```
Caja de curso -> assets/cursos.js -> GET cursos.php?term=Cis
-> SELECT nombre FROM cursos ... -> JSON -> menú de sugerencias
```

5 Estilos y bibliotecas utilizadas

assets/app.css

Contiene los estilos propios: ancho máximo del panel y de los formularios, alineación de las celdas, fondo de cada curso y presentación del menú de sugerencias. También define el elemento accesible que jQuery UI utiliza para anunciar resultados.

Para explicarlo: Bootstrap proporciona el diseño general y este archivo ajusta los componentes concretos del proyecto. Sus selectores corresponden a clases del HTML o a elementos creados por el autocompletado.

assets/bootstrap.min.css

Es la hoja de estilo de Bootstrap 5.3.8. Sus clases se usan en tablas, formularios, botones, alertas, contenedores, bordes, espaciados y columnas. Se carga desde head.php en todas las pantallas.

Para explicarlo: Permite aplicar un diseño uniforme y adaptable. No se carga JavaScript de Bootstrap porque la aplicación no utiliza componentes que lo necesiten.

assets/jquery.min.js

Es jQuery 3.7.1. Se carga en el formulario antes de jQuery UI y cursos.js. Proporciona la selección de elementos con $(), eventos con on(), inserción y eliminación de filas, y llamadas AJAX.

Para explicarlo: El código propio de cursos.js depende de jQuery. Por eso este archivo sí es necesario y no debe borrarse durante la limpieza.

assets/jquery-ui.min.js

Es jQuery UI 1.14.1. Proporciona el widget autocomplete() y sus dependencias internas, como el menú de sugerencias y el manejo del teclado. Se carga solo en el formulario de alumnos.

Para explicarlo: Es una biblioteca de terceros distribuida como paquete. Su uso concreto aquí es el autocompletado. Se conserva con su licencia, igual que Bootstrap y jQuery; no es un archivo de prueba ni una segunda implementación propia.

Por qué los recursos son locales

Las cinco hojas y scripts de assets se sirven desde WAMP. Después de copiar el proyecto, el diseño y la interacción no necesitan descargar bibliotecas de Internet. El servidor Apache y MySQL sí deben estar activos para que PHP y las sugerencias funcionen.

Orden de carga

```
head.php: bootstrap.min.css -> app.css
alumno_form.php: jquery.min.js -> jquery-ui.min.js -> cursos.js
```

El orden importa: cursos.js usa funciones que deben existir al ejecutarse. No se dejaron bibliotecas de Bootstrap JS, hojas de tema antiguas ni otra versión de jQuery cargada en paralelo.

6 Modelo de datos y relaciones

La base alumnos usa MySQL con InnoDB y utf8mb4. La organización sigue las cuatro entidades del ER. Los textos tienen intercalación utf8mb4_unicode_ci, que compara nombres sin distinguir mayúsculas y puede considerar equivalentes variantes con acentos.

usuario

Campos: id_usuario INT autoincremental, correo VARCHAR(30) único, nombre VARCHAR(35) y password VARCHAR(255). Su clave primaria es id_usuario. La contraseña se almacena como hash; el acceso se verifica con password_verify().

alumno

Campos: id_alumno INT autoincremental, nombres VARCHAR(50), apellidos VARCHAR(50), correo VARCHAR(30), aficiones VARCHAR(100) e id_usuario INT. La clave foránea id_usuario apunta al usuario asociado cuando se crea el registro. La aplicación permite a cualquier usuario autenticado editar o eliminar registros; no aplica permisos por propietario.

cursos

Campos: id_curso INT autoincremental y nombre VARCHAR(50) único. Es el catálogo compartido. Un nombre ya existente se reutiliza en lugar de crear otra fila con el mismo nombre.

alumno_cursos

Campos: id_alumno INT, id_curso INT y anio SMALLINT UNSIGNED. Su clave primaria está formada por los tres campos. Permite asignar varios cursos a un alumno y registrar el mismo curso en años diferentes. La regla CHECK admite años de 1900 a 2100.

Relación	Significado
usuario 1 a N alumno	Un usuario puede estar asociado a varios alumnos.
alumno 1 a N alumno_cursos	Un alumno tiene varias asignaciones; al borrarlo se eliminan esas asignaciones.
cursos 1 a N alumno_cursos	Un curso puede estar asignado a distintos alumnos o años.

Ajustes necesarios respecto al dibujo

El ER dibujaba anio como TINYINT, que no admite 2026; por eso se utiliza SMALLINT UNSIGNED. El campo password no tenía longitud definida en el dibujo; se usa VARCHAR(255) para admitir hashes. Se mantienen los nombres, las longitudes de los demás textos y las relaciones del modelo.

Para explicarlo: alumno_cursos resuelve la relación entre muchos alumnos y muchos cursos y añade el año de cada asignación. El nombre del curso se consulta desde cursos, sin repetirlo en cada asignación.

7 Restaurar la base desde el SQL

database.sql

Es el respaldo exportado de MySQL con estructura y contenido. Incluye CREATE DATABASE y USE para alumnos, CREATE TABLE, claves primarias y foráneas, índices, regla del año, valores AUTO_INCREMENT e INSERT de los registros actuales. Conserva el hash del usuario.

Para explicarlo: Importar este único archivo permite instalar la base sin ejecutar un instalador PHP. Es una fotografía de la base al exportarla; si después se modifican registros, el archivo no se actualiza por sí solo.

Tabla	Registros incluidos
usuario	1
alumno	1
cursos	1
alumno_cursos	1

Pasos con phpMyAdmin o MySQL Workbench

1. Iniciar Apache y MySQL de WAMP. Conectarse a MySQL en 127.0.0.1, puerto 3306, con root y contraseña vacía.

2. En un equipo de destino sin las tablas del proyecto, importar database.sql completo. En phpMyAdmin puede hacerse desde Importar a nivel del servidor. En Workbench puede abrirse el archivo SQL y ejecutarse completo.

3. Comprobar que exista alumnos con sus cuatro tablas y los datos indicados. El archivo ya contiene la creación y selección de la base.

4. Copiar el proyecto al directorio web de WAMP y abrir http://localhost/php_js_jquery/. Ingresar con admin@alumnos.local y la contraseña inicial Alumno2026!.

Alcance de la importación

El SQL está preparado para una instalación nueva. No contiene DROP DATABASE ni DROP TABLE y no es un script para mezclar o actualizar tablas existentes. Si ya existen las tablas, MySQL informará que existen; no se debe ejecutar sobre la base en uso para intentar duplicarla.

Las líneas de configuración y los comentarios especiales /*! ... */ fueron generados por MySQL. Configuran temporalmente codificación y comprobaciones durante la restauración y después recuperan los valores de sesión. No son bloques PHP comentados sin uso.

Comprobación realizada

Se importó el SQL en una base temporal con nombre diferente y se compararon los datos de las cuatro tablas, sus estructuras equivalentes, claves y contadores con alumnos. La verificación fue satisfactoria y se eliminó únicamente la base temporal.

8 Limpieza y guion para presentar

Código que se retiró

consulta.php: era un alias de ver.php sin enlaces de uso. Se eliminó. **edit.php:** se consolidó en editar_db.php y se actualizó el enlace de la lista. Ya no hay dos rutas para editar. **instalar.php:** se eliminó porque database.sql restaura la base y el usuario completos.

Se quitó de alumno_form.php una comprobación de sesión repetida: nuevo.php y editar_db.php ya la realizan antes de abrir la conexión. Se conservaron las validaciones, las transacciones y las protecciones CSRF porque forman parte del comportamiento de los formularios. Las pruebas auxiliares quedaron fuera de la carpeta de código entregable.

Otros archivos de la entrega

README.md: instrucciones breves de instalación, acceso, archivos y limpieza. **ER.xlsx:** modelo y bosquejos de referencia; no es código ejecutable. **output/pdf/guia_del_proyecto.pdf:** este documento para estudiar y explicar el funcionamiento; tampoco se ejecuta como parte del sitio.

Demostración sugerida

1. Abrir la lista sin sesión: mostrar Ingresar, Modo lectura y Ver. Intentar entrar a editar_db.php sin sesión y explicar el regreso al login.

2. Ingresar y mostrar Nuevo, Editar y Eliminar. Crear un alumno, presionar + varias veces y quitar una fila con el botón de eliminación.

3. Escribir el comienzo de un curso existente para mostrar las sugerencias. Guardar y consultar alumno y alumno_cursos en MySQL.

4. Editar desde el enlace de la fila, modificar un dato y los cursos, guardar y comparar la lista con MySQL. Para la demostración de eliminación, usar un registro de prueba y confirmar su borrado.

5. Cerrar sesión y mostrar que las acciones protegidas vuelven a bloquearse.

Preguntas que conviene saber responder

¿Por qué hay un formulario compartido? Para evitar duplicar campos, reglas y guardado entre alta y edición. **¿Por qué una transacción?** Para confirmar alumno y cursos juntos o revertir ambos si hay un error.

¿Por qué un token CSRF? Para comprobar que la solicitud incluye el token de la sesión. **¿Por qué escapar HTML?** Para que el texto del usuario no se interprete como etiquetas o scripts. **¿Por qué no borrar cursos al eliminar un alumno?** Porque pertenecen al catálogo común y pueden seguir usándose.